

SC1001: Introduction to Computational Thinking

Week 4: Repetition (Looping)

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

`boydgraber.org/teaching/SC_1001/`

Overview

1. Count temperatures above a threshold with a loop
2. Trace a loop with `continue`, `pass`, and `while-else`
3. Repair a sentinel loop and extend it to track more information
4. Write a looping solution for FizzBuzz
5. Print a pyramid pattern with `for` loops

Discussion 1: Days Above 25°C

Assume we are already given a list called `daily_temperatures` with 30 values.

Write Python code that counts how many *days* have temperature strictly above 25.

```
daily_temperatures = [22, 25, 27, 23, 28, 26, 24, 30, 31, 29,  
                      21, 22, 26, 27, 28, 29, 25, 30, 31, 32,  
                      20, 21, 22, 23, 24, 25, 26, 27, 28, 29]
```

Discussion 1: Add-On

Suppose we instead store the temperatures by week:

```
temperature_by_week = {  
    "week 1": [22, 25, 27, 23, 28, 26, 24],  
    "week 2": [30, 31, 29, 21, 22, 26, 27],  
    "week 3": [28, 29, 25, 30, 31, 32, 20],  
    "week 4": [21, 22, 23, 24, 25, 26, 27],  
    "week 5": [28, 29]  
}
```

How would that change the solution?

Discussion 2: What Is the Output?

```
value = 6
if value % 2 == 0:
    print("first", value)
elif value % 3 == 0:
    print("second", value)
while value <= 9:
    value = value + 1
    if value == 8:
        continue
    else:
        pass
    print("third", value)
else:
    print("fourth", value)
print("fifth", value)
```

Discussion 2: What Is the Output?

```
value = 6
if value % 2 == 0:
    print("first", value)
elif value % 3 == 0:
    print("second", value)
while value <= 9:
    value = value + 1
    if value == 8:
        continue
    else:
        break
    print("third", value)
else:
    print("fourth", value)
print("fifth", value)
```

Discussion 3A: Find and Fix the Errors

The program should:

- count how many input strings contain the letter a,
- stop when the user enters ####,
- and print the total at the end.

```
while True:
    str = input("enter a string: ")
    for letter in str:
        if letter == 'a':
            break
    count += 1
print(count, "strings with letter 'a'")
```

Discussion 3B: Membership Version

Can we solve the repaired problem using direct string membership instead of an inner loop over letters?

Discussion 3C: List-Comprehension Version

Can we use a list comprehension to print the total number of a's entered across all input strings?

Discussion 4: Write the Python Program

Write a looping solution for FizzBuzz from 1 to 20.

- print FizzBuzz for multiples of both 3 and 5
- otherwise print Fizz for multiples of 3
- otherwise print Buzz for multiples of 5
- otherwise print the number

Discussion 5A: Print the Pattern

Read an integer width and print a pattern.

For example, if the input is 4, print

```
*  
**  
***  
****  
***  
**  
*
```

Start with nested for loops.

How many total numbers do we print?

How many total numbers do we print?

$$\sum_{i=1}^{i \leq n} i = \frac{n(n+1)}{2} \quad (1)$$

How many total numbers do we print?

$$\sum_{i=1}^{i \leq n} i = \frac{n(n+1)}{2} \quad (1)$$

Sketch of proof (attributed to Gauss): if $n = 100$ can pair $(1, 100)$, $(2, 99)$, etc. every pair, always adds to $n + 1$. There are $\frac{n}{2}$ of them.

How many total numbers do we print?

$$\sum_{i=1}^{i \leq n} i = \frac{n(n+1)}{2} \quad (1)$$

Sketch of proof (attributed to Gauss): if $n = 100$ can pair $(1, 100)$, $(2, 99)$, etc. every pair, always adds to $n + 1$. There are $\frac{n}{2}$ of them.

But we're summing 1 to n and $n - 1$ to 1, so we have a total of:

$$\sum_{i=1}^{i \leq n} + \sum_{i=1}^{i < n} = \frac{n(n+1)}{2} + \frac{(n-1)n}{2} \quad (2)$$

$$= \frac{n^2}{2} + \frac{n}{2} + \frac{n^2}{2} - \frac{n}{2} \quad (3)$$

$$= n^2 \quad (4)$$

Discussion 5B: Fewer Loops

Can you produce the same left-aligned pattern with only **two** for loops total?

Discussion 5C: Bonus

Bonus: can you produce the pattern with only **one** for loop?

Discussion 5D: Bonus Centered Version

Bonus: what if we want a centered pattern instead?

```
  *  
 * *  
* * *  
* * * *  
 * * *  
  * *  
   *
```

Takeaways

- Loops become easier to read when iterator names describe the role they play.
- Sentinel loops separate the stopping condition from the body of the computation.
- `continue` skips the rest of the current iteration, while `break` exits the loop entirely.
- Python has several equivalent styles: direct loops, membership tests, and comprehensions.
 - ▶ We're focusing on loops now because that's how underlying low-level code works
 - ▶ You should be able to read code with more advanced syntax
 - ▶ And know how expensive those operations are